

PROIECT CURS - Algoritmi Paraleli si Distribuiti
Procesarea Paralela a Imaginilor

Giotina Daria Maria

1.TEMA PROIECTULUI

Tema:

Procesarea de imagini (Image Processing)

Filtrarea, redimensionarea si transformarea imaginilor pot fi paralelizate pentru a procesa pixelii simultan, imbunatatind semnificativ performanta.

Variante implementate:

- Varianta secventiala (S5) : SequentialImageProcessor.java
- Varianta paralela #1 (S8) : ParallelImageProcessor.java

Limbaj de programare:

Java (versiunea 19.0.2, Java HotSpot(TM) 64-Bit Server VM)

Algoritmi implementati:

1) Gaussian Blur (filtru de netezire 3x3)

Aplica un kernel Gaussian pe fiecare pixel:

$$\frac{1}{16} * \begin{vmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{vmatrix}$$

Rezultat: imagine netezita, zgomot redus.

2) Conversie Grayscale

Formula ITU-R BT.601:

$$\text{gray} = 0.299 * R + 0.587 * G + 0.114 * B$$

Rezultat: imagine in tonuri de gri.

3) Detectie de margini Sobel

Aplica doua kerneluri Sobel (Gx si Gy):

$$\begin{matrix} \text{Gx: } \begin{vmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{vmatrix} & \text{Gy: } \begin{vmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{vmatrix} \end{matrix}$$

Magnitudine: $\sqrt{Gx^2 + Gy^2}$, clamped la 255.

Rezultat: contururile din imagine.

Tehnica de paralelizare (Varianta paralela #1):

- Framework: Java Fork/Join (java.util.concurrent)
- Clase folosite: ForkJoinPool, RecursiveAction
- Strategie: descompunerea domeniului pe linii (domain decomposition)

Imaginea este impartita in benzi orizontale de pixeli.

Fiecare banda este procesata de un task independent.

Daca banda e prea mare ($> \text{THRESHOLD} = 64$ linii), se divide recursiv in doua subtask-uri prin fork/join (divide and conquer).

- Pool: ForkJoinPool cu $p = \text{availableProcessors}()$ fire de executie.

- Fara sincronizare: fiecare task scrie in zone disjuncte de memorie.

Fisiere sursa:

src/Main.java - benchmark comparativ + validare

src/SequentialImageProcessor.java - varianta secventiala

src/ParallelImageProcessor.java - varianta paralela #1 (Fork/Join)

src/ImageUtils.java - utilitare: load, save, generate, compare

Marimea imaginii de test: 4000 x 3000 pixeli (12.000.000 pixeli)

Numar de rulari pentru medie: 3 (+ 1 warmup pentru JIT)

2. INFORMATII DESPRE MASINA PE CARE S-A RULAT CODUL

Sistem de operare: Windows 11 (NT 10.0 Build 26200)

Procesor: Intel Core i7-12700H (12th Gen)

Nuclee fizice: 14

Fire logice (p): 20

Frecventa maxima: 2000 MHz (boost pana la 4700 MHz)

Memorie RAM: 16 GB

JVM: Java HotSpot 64-Bit Server VM

Versiune Java: 19.0.2 (build 19.0.2+7-44, mixed mode, sharing)

Parallelism folosit: 20 fire (ForkJoinPool cu $\text{availableProcessors}()$)

3. REZULTATE EXPERIMENTALE

Imagine de test: generata procedural (gradient RGB), 4000 x 3000 px

Rulari: 3 (media timpilor, dupa warmup JIT)

TEST 1: Gaussian Blur (filtru 3x3, convolutie pe 9 vecini)

Timp secvential (T_s): 599.69 ms

Timp paralel (T_p): 261.25 ms

Accelerare ($S = T_s/T_p$): 2.30x

Eficienta ($E = S/p$): 0.11 (ideal = 1.0)

Cost suplimentar ($T_o = p \cdot T_p - T_s$): 4625.41 ms

TEST 2: Conversie Grayscale

Timp secvential (T_s): 193.38 ms

Timp paralel (T_p): 141.28 ms

Accelerare ($S = T_s/T_p$): 1.37x
Eficienta ($E = S/p$): 0.07 (ideal = 1.0)
Cost suplimentar ($T_o = p \cdot T_p - T_s$): 2632.19 ms

TEST 3: Detectie de margini Sobel (grayscale + dubla convolutie)

Timp secvential (T_s): 1155.54 ms
Timp paralel (T_p): 460.68 ms
Accelerare ($S = T_s/T_p$): 2.51x
Eficienta ($E = S/p$): 0.13 (ideal = 1.0)
Cost suplimentar ($T_o = p \cdot T_p - T_s$): 8058.07 ms

VALIDARE REZULTATE

Gaussian Blur : secvential == paralel -> DA (identice)
Grayscale : secvential == paralel -> DA (identice)
Sobel : secvential == paralel -> DA (identice)

INTERPRETARE REZULTATE

Accelerare obtinuta: intre 1.37x (Grayscale) si 2.51x (Sobel),
pe o masina cu $p = 20$ fire logice.

Eficienta scazuta (0.07 - 0.13) se datoreaza in principal
costului de sincronizare al API-ului BufferedImage:
metodele `getRGB()` / `setRGB()` sunt sincronizate intern in Java,
creand contention intre firele de executie.

Sobel obtine cea mai buna accelerare deoarece are cel mai mare
volum de calcul per pixel (doua convolutii + sqrt), deci raportul
calcul / acces memorie este mai favorabil paralelizarii.

Conform Legii lui Amdahl: $S = 1 / (\sigma + (1 - \sigma) / p)$
Portiunea secventiala σ (overhead JVM + I/O imagine) limiteaza
accelerarea maxima posibila indiferent de numarul de procesoare.